

LeetCode Deep Dive

From First Instinct to Optimal Algorithm: Two-Dimensional DP in Practice
Week 9 Supplement – TOAE209/TOAH209: Design and Analysis of Algorithms

Foreign Trade University

Outline

Why walk through LeetCode problems?

- This week's LEETCODE.md list gives ten problems to practice on your own. This deck **narrates the thinking process** for two of them end to end.
- We pick one **Medium** (*Edit Distance* – exactly this week's lecture problem) and one **Hard** (*Distinct Subsequences*) – both solved by the *same* two-parameter DP recurrence taught this week, applied to two different questions: “what is the **best** alignment?” vs. “**how many** alignments are there?”

The four-step process we will repeat twice

- ❶ **First instinct.** Write the brute-force recursion that follows directly from the problem statement – try every choice at every step.
- ❷ **Measure why it hurts.** Compute the actual size of the recursion tree and plug in the problem's real constraints.
- ❸ **Find the structural insight.** Identify the *overlapping subproblems* – the same (prefix-of- X , prefix-of- Y) pair recomputed exponentially many times – and memoize on exactly that pair.
- ❹ **Implement the optimal algorithm,** hand-trace it on a small example, then look for programming-level (not algorithmic) implementation tricks.

This mirrors the course

Step 3 *is* this week's core lesson: a DP recurrence is nothing more than “memoize the brute-force recursion on its real parameters.”

Problem statement

LeetCode 72 – Edit Distance (Medium)

Given two strings word1 and word2, return the minimum number of single-character **insert**, **delete**, or **replace** operations needed to turn word1 into word2.

- This *is* this week's Levenshtein edit distance: $\delta = 1$ (gap cost) and $\alpha_{pq} = 1$ for $p \neq q$ (mismatch cost).
- Constraints (LeetCode): $0 \leq |\text{word1}|, |\text{word2}| \leq 500$.

Worked example

word1 = "horse", word2 = "ros" $\rightarrow$ answer **3**.

horse	$\rightarrow$	rorse	$\rightarrow$	rose	$\rightarrow$	ros
	replace h $\rightarrow$ r		delete r		delete e	

- Three operations suffice: replace h $\rightarrow$ r, delete the second r, delete the trailing e.
- Is 3 really the *minimum*? Nothing above proves it – we need the DP to certify optimality, which is exactly what the recurrence below does.

First instinct: brute-force recursion

Most direct reading of the problem

To convert `word1[0:i]` into `word2[0:j]`: if the last characters match, recurse on both shorter prefixes for free; otherwise, try all three operations on the last character and take the best.

```
1: function EDIT(i,j)
2:   if i = 0 then return j
3:   else if j = 0 then return i
4:   end if
5:   if word1[i] = word2[j] then
6:     return EDIT(i - 1, j - 1)
7:   else
8:     return 1 + min (EDIT(i - 1, j), EDIT(i, j - 1), EDIT(i - 1, j - 1))
9:   end if
10: end function
```

Why this is bad

- Every mismatched pair branches into (up to) 3 recursive calls, each on strings one shorter.
- Worst case (two totally mismatched strings): the recursion tree has up to $3^{\min(n,m)}$ leaves.
- Plug in real numbers: two 20-character strings $\Rightarrow$ up to $3^{20} \approx 3.49$ billion recursive calls
 - for strings LeetCode allows up to length **500**, this is astronomically infeasible.

The smell to notice

`EDIT(i, j)` gets called with the *same* (*i*, *j*) pair over and over from different branches of the recursion tree – the function has only $(n+1)(m+1) \leq 501^2$ possible distinct inputs, yet the raw recursion revisits them billions of times.

The structural insight

- **Overlapping subproblems:** there are only $(n + 1)(m + 1)$ distinct (i, j) pairs the recursion can ever be called on. Memoize the answer for each pair the first time it is computed, and every later call is $O(1)$.
- **Optimal substructure:** once you fix what happens to the last characters, the rest of an optimal alignment is provably an optimal alignment of the two shorter prefixes – exactly the argument from this week's lecture.
- Together these give $O(nm)$ time and space: fill a table bottom-up instead of recursing top-down, in whichever order guarantees $dp[i - 1][\cdot]$ and $dp[\cdot][j - 1]$ are ready before $dp[i][j]$.

Optimal algorithm: bottom-up DP

```
1: function EDITDISTANCE(word1, word2)
2:    $n, m \leftarrow |word1|, |word2|$ 
3:    $dp[i][0] \leftarrow i$  for  $i = 0, \dots, n$ ;  $dp[0][j] \leftarrow j$  for  $j = 0, \dots, m$ 
4:   for  $i \leftarrow 1$  to  $n$  do
5:     for  $j \leftarrow 1$  to  $m$  do
6:       if  $word1[i] = word2[j]$  then
7:          $dp[i][j] \leftarrow dp[i-1][j-1]$ 
8:       else
9:          $dp[i][j] \leftarrow 1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])$ 
10:      end if
11:    end for
12:  end for
13:  return  $dp[n][m]$ 
14: end function
```

$O(nm)$ time and space – exactly this week's bound.

Trace on the worked example

word1 = "horse" (rows), word2 = "ros" (columns). $dp[i][j]$ = edit distance between the first i letters of horse and first j of ros.

	" "	r	o	s
" "	0	1	2	3
h	1	1	2	3
o	2	2	1	2
r	3	2	2	2
s	4	3	3	2
e	5	4	4	3

Bottom-right cell $dp[5][3] = 3$ matches the answer. E.g. $dp[2][2] = 1$ because h,o vs. r,o: last letters o=o match, so $dp[2][2] = dp[1][1] = 1$.

Complexity: naive vs. optimal

Approach	Time	At $n = m = 500$
Brute-force recursion	$O(3^{\min(n,m)})$	incomputable
Memoized / bottom-up DP	$O(nm)$	250,000 ops

The DP visits each of the $(n+1)(m+1)$ table cells exactly once, doing $O(1)$ work per cell – the entire saving comes from never solving the same subproblem twice.

Python implementation

```
1 def minDistance(word1, word2):
2     n, m = len(word1), len(word2)
3     dp = [[0] * (m + 1) for _ in range(n + 1)]
4     for i in range(n + 1):
5         dp[i][0] = i
6     for j in range(m + 1):
7         dp[0][j] = j
8
9     for i in range(1, n + 1):
10        for j in range(1, m + 1):
11            if word1[i - 1] == word2[j - 1]:
12                dp[i][j] = dp[i - 1][j - 1]
13            else:
14                dp[i][j] = 1 + min(dp[i - 1][j],          # delete
15                                   dp[i][j - 1],          # insert
16                                   dp[i - 1][j - 1])        # replace
17
18    return dp[n][m]
```

Implementation tips & tricks (the programming side)

- **Table size is $(n+1) \times (m+1)$, not $n \times m$.** The extra row and column hold the “one string is empty” base cases; forgetting the +1 is the single most common off-by-one bug in this problem.
- **Rolling 1-D array.** $dp[i][\cdot]$ only ever depends on row $i - 1$, so you can drop the table from $O(nm)$ to $O(\min(n, m))$ space (swap the strings so the shorter one indexes the array) – a pure memory optimization, it does not change the $O(nm)$ time.
- **Recursive memoization (`@lru_cache`) works but costs more in practice** than the flat array above: each call pays dict-hashing/function-call overhead, and strings longer than Python's recursion limit (≈ 1000) will crash a naive recursive version outright – prefer the iterative table for anything performance-sensitive.

Problem statement

LeetCode 115 – Distinct Subsequences (Hard)

Given strings s and t , return the number of distinct **ways** t occurs as a subsequence of s (count by which positions of s you pick, not by the resulting string value).

- This is the exact same “align a prefix of X with a prefix of Y ” idea as Edit Distance – but instead of asking for the *best* alignment, it asks you to **count** all valid ones.
- Constraints (LeetCode): $|s| \leq 1000$, $|t| \leq 1000$.

Worked example

$s = \text{"aab"}, t = \text{"ab"} \rightarrow \text{answer } 2.$

- Positions of s (0-indexed) a a b: pick index 0 (a) and index 2 (b) $\rightarrow$ spells "ab".
- Pick index 1 (a) and index 2 (b) $\rightarrow$ also spells "ab".
- Two *different* choices of positions both spell t , so the answer counts **2** – even though the resulting string is the same both times.

First instinct: try every subset of positions

Most direct reading of the problem

Try every subset of positions of s ; keep it if the characters at those positions, read in order, spell exactly t ; count how many subsets qualify.

```
1:  $count \leftarrow 0$ 
2: for each of the  $2^{|s|}$  subsets  $S$  of positions of  $s$  do
3:   if characters of  $s$  at positions  $S$ , in order, equal  $t$  then
4:      $count \leftarrow count + 1$ 
5:   end if
6: end for
7: return  $count$ 
```

Why this is bad

- There are $2^{|s|}$ subsets, and checking each one against t costs $O(|t|)$.
- Plug in real numbers: LeetCode allows $|s|$ up to 1000 $\Rightarrow 2^{1000}$ subsets – a number with over 300 digits, far beyond the $\sim 10^{80}$ atoms in the observable universe (the same futility we saw with $\log^* n$ in Week 3's supplement, but here the blow-up is the search space itself, not a growth rate).

The smell to notice

Once again: the recursive version of “does a match starting here work?” only ever gets called with a pair (position in s , position in t) – just $(|s|+1)(|t|+1) \leq 1001^2$ distinct pairs – yet brute force revisits them an astronomical number of times.

The structural insight

Let $dp[i][j]$ = number of ways $t[0:j]$ occurs as a subsequence of $s[0:i]$. Reason about s_i , the last character of the current prefix of s :

- s_i can always be **skipped** (not used in the match): this contributes $dp[i-1][j]$ ways, regardless of what s_i is.
- If $s_i = t_j$, s_i can *additionally* be **used** to match t_j : this contributes $dp[i-1][j-1]$ more ways.

$$dp[i][j] = dp[i-1][j] + [s_i = t_j] \cdot dp[i-1][j-1], \quad dp[i][0] = 1, \quad dp[0][j > 0] = 0.$$

This is the *same* recurrence shape as Edit Distance's "last character" case analysis – sum instead of min, because we are counting instead of optimizing.

Trace on the worked example

$s = \text{"aab"}$ (rows), $t = \text{"ab"}$ (columns). $dp[i][j]$ = number of ways $t[0:j]$ is a subsequence of $s[0:i]$.

	" "	a	b
" "	1	0	0
a	1	1	0
a	1	2	0
b	1	2	2

Row 3 ($s[2]=\text{"b"}$), column "b": $s_3 = t_2 = \text{b}$, so $dp[3][2] = dp[2][2] + dp[2][1] = 0 + 2 = 2$ – matches the expected answer.

Optimal algorithm: pseudocode

```
1: function NUMDISTINCT( $s, t$ )
2:    $n, m \leftarrow |s|, |t|$ 
3:    $dp[i][0] \leftarrow 1$  for  $i = 0, \dots, n$ ;  $dp[0][j > 0] \leftarrow 0$ 
4:   for  $i \leftarrow 1$  to  $n$  do
5:     for  $j \leftarrow 1$  to  $m$  do
6:        $dp[i][j] \leftarrow dp[i-1][j]$ 
7:       if  $s_i = t_j$  then
8:          $dp[i][j] \leftarrow dp[i][j] + dp[i-1][j-1]$ 
9:       end if
10:    end for
11:  end for
12:  return  $dp[n][m]$ 
13: end function
```

$O(nm)$ time and space – versus $O(2^n \cdot m)$ for brute force.

Complexity: naive vs. optimal

Approach	Time	At $ s = 1000$, $ t = 100$
Try every subset of s	$O(2^{ s } \cdot t)$	incomputable
Bottom-up DP	$O(s t)$	100,000 ops

Python implementation

```
1 def numDistinct(s, t):
2     n, m = len(s), len(t)
3     dp = [[0] * (m + 1) for _ in range(n + 1)]    # NOT [[0]*(m+1)]*(n+1) !
4     for i in range(n + 1):
5         dp[i][0] = 1                               # empty t: exactly one way
6
7     for i in range(1, n + 1):
8         for j in range(1, m + 1):
9             dp[i][j] = dp[i - 1][j]                # always allowed: skip s[i-1]
10            if s[i - 1] == t[j - 1]:
11                dp[i][j] += dp[i - 1][j - 1]        # also allowed: use s[i-1]
12    return dp[n][m]
```

Implementation tips & tricks (the programming side)

- `[[0]*(m+1) for _ in range(n+1)]`, **never** `[[0]*(m+1)]*(n+1)`. The second form creates $n+1$ references to the *same* inner list – mutating one row silently mutates every row. This is one of the most common real Python bugs in 2-D DP, and it compiles/runs without any error, it just silently gives the wrong answer.
- **Counting DP has no overflow issue in Python** (unlike C/C++/Java, where the count can need 64-bit or even bignum arithmetic for large $|s|$) – but note that LeetCode's official constraints keep the true answer within signed 32-bit range specifically *because* other languages need to represent it.
- **If space-optimizing to a 1-D rolling array**, you must update j in *increasing* order (each new value needs the *old*, not-yet-overwritten $dp[j-1]$ from the previous row) – the *opposite* direction from the right-to-left rule you may know from 0/1-knapsack space optimization. Mixing the two conventions up is a genuinely common bug.

Summary: two problems, one recurrence

- 1 **Edit Distance** asks for the *best* way to align two prefixes; **Distinct Subsequences** asks *how many* ways a prefix of t embeds in a prefix of s . Both recurrences reason about “what happens to the last character,” exactly this week’s recipe – one takes a $\min$, the other takes a $+$.
- 2 Both naive instincts were the same shape: brute-force enumeration (all edit sequences; all position subsets) that revisits a tiny number of distinct (i, j) subproblems an exponential number of times.
- 3 Both fixes were the same shape too: notice the overlap, memoize on (i, j) , get $O(nm)$.
- 4 The implementation tricks (rolling arrays, list-of-lists aliasing, the knapsack-vs-subsequence rolling-array direction) are a second, independent layer – the *algorithm* can be optimal while the *code* is still subtly wrong.

Keep practicing

LEETCODE.md lists eight more Week 9 problems (*Longest Common Subsequence*, *Regular Expression Matching*, *Unique Paths*, *Cheapest Flights Within K Stops*, ...) – try the same four-step process on each.